

Freshness as a First Class Schema Concern: Modeling Data Staleness Across 63 Live Sources

By Josie Leung, founder of Scuba Season

I am not a developer. I built Scuba Season, a nonprofit reef atlas covering 356 dive sites across 114 locations, using AI agents and no engineering background. The hardest problem I ran into was not the ocean science and it was not the frontend. It was staleness. Specifically, how do you build a schema that tells the truth about how old each piece of data actually is, when you are stitching together 63 sources that update on 63 different schedules, some daily, some weekly, some once a decade.

I want to walk through how that problem showed up in the codebase, because I think it is a pattern most data heavy apps hit and most of them get wrong.

The lie that is easy to tell by accident

Early on, every card in the atlas just showed a number. Coral cover: 32 percent. Fishing pressure: high. A user looking at that card has no way to know if the coral cover number is from a survey last month or a survey from 2010. Both render identically. Both feel equally current. That is the trap. A UI that shows data without provenance is quietly asserting that all data is equally fresh, and for us that assertion was false in a way that mattered. A dive site can look Thriving on a stale 2010 baseline and be Witnessing Change today.

So the real fix was not a UI polish pass, it was a schema decision: freshness needed to be a first class field on every data record, not a debugging afterthought.

Three data shapes, not one

Once I actually mapped our 63 sources with an agent, they sorted into 3 distinct freshness shapes, and each one needed its own contract.

Live. Data with an automated ingest running on a schedule, where “updated” has a real, checkable timestamp. NOAA Coral Reef Watch thermal stress data refreshes daily at 06:30 UTC through a GitHub Actions cron job, no API key required, which made it the cleanest source to model against. Global Fishing Watch fishing pressure and IUCN Red List species status update weekly. For this shape, the schema stores an ISO timestamp and the UI is allowed to say the word “live,” because it is actually true.

Snapshot. Data from a real survey with a real date attached, but no automated pipeline behind it, because the source organization itself does not publish on a schedule. Coral cover percentages fall here. NCRMP and AGRRA, the 2 major reef survey bodies we cite, do not expose an API. Their numbers update when a report gets published, not on any cadence we control. For most of our locations that means exactly 2 data points exist: a baseline around 2010 and a current reading from 2024. Not a trend line. A before and after. The schema has to carry a surveyDate and the UI has to show how many years old that survey actually is, because a 2 year old survey and a 14 year old survey should not look the same on the page.

Presence. Data that confirms a species was observed somewhere, sourced from GBIF and OBIS, but carries no freshness claim and no population trend at all. It just says: this animal has been recorded here. Treating a presence record like a trend measurement would be a second version of the same lie, so it gets its own visual treatment entirely.

What this looks like as an actual component

The pattern that made this maintainable was building one shared component, `DataFreshnessLabel`, with a discriminated union type instead of 3 different optional props bolted onto one interface.

```
type LiveProps = CommonProps & {
  variant: "live";
  source?: string;
  updatedAt?: string;
};

type SnapshotProps = CommonProps & {
  variant: "snapshot";
  surveyMethod: string;
  surveyDate?: string;
};

type PresenceProps = CommonProps & {
  variant: "presence";
  source?: string;
};

export type DataFreshnessLabelProps = LiveProps |
  SnapshotProps | PresenceProps;
```

The discriminated union does the enforcement work that a code review would otherwise have to do by hand. You cannot render a snapshot pill without a `surveyMethod`. You cannot render a live pill and forget to ask where the timestamp came from. The type system makes the freshness claim mandatory at the call site, not optional, and that is the whole point. If freshness is optional, it gets skipped under deadline pressure, and the card goes back to lying by omission.

Each variant also gets its own color and its own copy, on purpose. Live is emerald with a pulse dot. Snapshot is amber, and if the survey is more than 2 years old the component computes that itself and appends “(X years ago)” directly onto the label, so the staleness is not something a reader has to go dig for.

```
function yearsAgo(iso?: string): number | null {
  if (!iso) return null;
  const d = new Date(iso.length === 10 ? iso +
    "T00:00:00Z" : iso);
  if (Number.isNaN(d.getTime())) return null;
```

```
const years = (Date.now() - d.getTime()) / (365.25 *  
    24 * 3600 * 1000);  
return Math.floor(years);  
}
```

Freshness has to reach the classification logic too, not just the label

The label solves the display problem. It does not solve the harder problem, which is that our core feature, classifying every reef as Improving, Stable, or Declining, is a derived value built on top of these mixed freshness inputs. The classification function pulls the worst recorded thermal stress alert level and the most recent coral cover reading, then applies thresholds:

```
if ((bestCover !== null && bestCover < 25) ||  
    alertRank >= 3) {  
    return "change";  
}
```

That single function is quietly reading from both a live daily feed (thermal stress) and a snapshot that might be 4 years stale (coral cover), and producing one confident looking label. If I had not separated freshness at the schema level first, this function would have no way to distinguish “coral cover crashed last month” from “coral cover was measured once in 2010 and we are still using that number.” Getting the freshness contract right upstream is what let the classification logic stay simple downstream. The complexity has to live somewhere. I would rather it live once, explicitly, in the schema, than get silently reguessed inside every function that touches the data.

The honest number, in the end

After auditing all 63 sources against this 3 shape model, the live count was smaller than I expected going in. Thermal stress, fishing pressure, IUCN status, and new site discovery run on real automated schedules. Coral cover is a snapshot everywhere, because the science itself does not move faster than a report cycle. Species sightings were, for a while, a snapshot that was quietly synthetic, meaning the backfill process had generated one plausible sighting per site to avoid

empty states, which is its own lesson about how staleness bugs can hide inside data that looks populated. That has since moved to a real weekly iNaturalist and GBIF ingest.

None of that would have surfaced without treating freshness as something the schema enforces, not something a human remembers to caveat in the copy. If you are building anything that blends live feeds with survey data, my honest recommendation is to model the freshness shape before you model the display. The display problem is easy once the schema tells the truth.

Scuba Season is a free, nonprofit reef atlas at scubaseason.fun.